

Como trocar o dataset — caminho alternativo da D1

D1 — Programação Python para IA · leia junto com o COMO-USAR-ESTE-PROJETO.md

Este documento é para quem quer usar um conjunto de dados **diferente** do padrão do pacote — dentro das alternativas que o curso já mapeou e testou. Nada aqui inventa fonte nova: tudo vem do MAPA-DE-DATASETS da disciplina e das nove receitas de download prontas que o acompanham.

1. Quando vale a pena e quando não vale

Trocar é **escolha, não mérito**: o caminho padrão e o caminho com dataset próprio valem a mesma nota, e a rubrica não distingue entre eles. Trocar faz sentido quando o aluno já sabe programar e quer que a fundação do curso trabalhe desde já com os dados do próprio domínio — a disciplina seguinte herda o dataset, e quem troca chega lá com a matéria-prima do próprio projeto. Não faz sentido trocar para "ganhar" alguma coisa: o que a avaliação cobra (repositório que clona e roda, código entendido e explicado, análise com evidência, honestidade com a fonte) é o mesmo nos dois casos, e a troca tem um custo real, listado na seção 4.

Trocar de caminho é permitido **nos dois sentidos, a qualquer momento e sem justificar** — a regra da disciplina para os dois caminhos com a mesma nota continua valendo aqui. Quem trocou e travou volta ao padrão sem perder nada; quem começou pelo padrão adota o dataset próprio quando quiser.

2. As alternativas que o curso mapeou

O MAPA-DE-DATASETS cobre os oito domínios do curso, com candidatos testados e, para a maioria, **receita de download pronta**:

Domínio	Receita pronta	Observação do mapa
Manutenção industrial	<code>baixar-dados-manutencao.py</code> (<code>--conjunto hidraulico</code> ou <code>secom</code>)	é o domínio do exemplo; o AI4I é o dataset do padrão
Suporte técnico de redes	<code>baixar-dados-suporte.py</code>	pede token gratuito do Kaggle
Jurídico/documental	<code>baixar-dados-juridico.py</code>	não precisa de conta — a chave pública do CNJ já vem na receita
Triagem de atendimento	<code>baixar-dados-triagem.py</code>	pede token do Kaggle
Compliance ambiental	<code>baixar-dados-compliance.py</code> (tabular) ou os PDFs das NRs (documental)	duas naturezas; o mapa explica
Crédito ou fraude	<code>baixar-dados-credito.py</code> / <code>baixar-dados-fraude.py</code>	fraude baixa 150,8 MB e amostra para 50 mil
Logística	<code>baixar-dados-logistica.py</code>	pede token do Kaggle

Domínio	Receita pronta	Observação do mapa
Saúde (administrativo)	<code>baixar-dados-saude.py</code>	agendamento, jamais diagnóstico

Os limites que o mapa fixa para qualquer escolha:

Limite	Valor
Download	até 200 MB (a receita recusa acima disso)
Linhas	até 50.000 (acima disso, a receita gera amostra sistemática)
Piso	≥ 500 registros ou ≥ 50 documentos
Corpus documental	até 200 documentos e 100 MB

A candidata que o mapa **não** lista para troca é justamente o AI4I 2020 do pacote: no mapa, escrito para a turma do caminho padrão, ele aparece marcado como "dataset do exemplo, não aceito para troca". Neste pacote a posição dele é outra — ele **é** o ponto de partida — e a diferença está explicada na seção 6.

3. O procedimento, passo a passo

1. **Rode a receita do seu domínio** a partir da raiz do repositório. As nove receitas já vêm neste pacote, na pasta `receitas/`, junto com o `MAPA-DE-DATASETS.pdf` — que diz qual é a receita do seu domínio e o que ela pede (só `pandas`, ou token do Kaggle no `.env`). A do domínio jurídico é a mais leve de testar: não precisa de conta nem de chave própria:

```
```powershell
```

```
python receitas\baixar-dados-juridico.py
```

```
```
```

A receita faz tudo sozinha: baixa respeitando o teto de tamanho, converte para o formato do curso, cria a proveniência e reclama se o resultado ficar abaixo do piso.

1. **O que nasce:** a pasta `dados/` na raiz, com `dados/dataset.csv` (ou `dados/documentos/` no domínio documental) e o `dados/FONTE.md` com URL, data do download, licença, resumo criptográfico e contagem de linhas. Esse `FONTE.md` é o que o `verifica-entrega.py` procura como prova de procedência — a receita o preenche; não edite à mão.
1. **O que fazer com o `data/` que veio no pacote:** nada precisa ser apagado. A amostra e as fixtures continuam servindo ao modo sem rede e aos testes; o código do projeto, porém, vai passar a ler o dataset novo — e é aí que mora o trabalho da seção 4. Quem quiser manter o repositório enxuto pode remover `data/raw/` (o download do padrão), desde que o `dados/` novo esteja no lugar.
1. **Confirme** com `python scripts\verifica-entrega.py`: o item de dataset deve apontar para a pasta `dados/` com a contagem nova (acima do piso).

4. O que quebra — dito sem rodeio

O código do pacote foi escrito para as colunas do AI4I e para as ordens de serviço da manutenção. Trocar o dataset **não é trocar um arquivo**: é adaptar a camada que dá nome às coisas. Arquivo por arquivo:

| Arquivo | O que acontece | O que fazer |
|--|---|---|
| <code>src/ams/dados.py</code> | quebra no primeiro <code>carregar()</code> : o mapa de colunas, os tipos e o validador são do AI4I; o caminho fixo aponta para <code>data/raw/ai4i2020.csv</code> | reescrever o mapa de colunas, os tipos e <code>caminho_dataset()</code> para o CSV novo |
| <code>src/ams/modelos.py</code> | o modelo <code>LeituraSensor</code> tem os campos do sensor de usinagem | reescrever o modelo com os campos do seu domínio (os demais modelos servem) |
| <code>src/ams/analise.py</code> | as agregações respondem perguntas de falha, desgaste e modos | reescrever as funções para as perguntas do seu domínio |
| <code>tests/test_dados.py</code> ,
<code>tests/test_analise.py</code> | testam o comportamento antigo — passam a falhar com razão | reescrever contra o comportamento novo (os demais arquivos de teste continuam) |
| <code>tests/conftest.py</code> | prepara registros no formato do AI4I | ajustar ao formato novo |
| <code>notebooks/01-eda.ipynb</code> | as oito perguntas são de manutenção; o notebook roda, mas responde o domínio errado | reescrever as perguntas (e as conclusões) para o seu dataset — mantendo a seção final do agente |
| <code>src/ams/cliente_llm.py</code> | <code>gerar_ordens()</code> lê colunas do AI4I e monta ordens de serviço de manutenção | adaptar <code>gerar_ordens()</code> e as descrições às entidades do seu domínio |
| <code>data/fixtures/</code> | as respostas gravadas foram feitas para as ordens da manutenção; ordens novas têm identificador novo e fixture inexistente — o modo sem rede falha com instrução | regravar, só depois do passo anterior |
| <code>scripts/baixar-dados.py</code> | baixa e valida o AI4I (a cascata confere as 14 colunas) | substituir pela receita do domínio (ou manter os dois, com nomes diferentes) |
| <code>data/FONTE.md</code> | descreve o AI4I | o <code>dados/FONTE.md</code> novo assume o papel de proveniência |

O que **continua funcionando sem mudança**: `config.py` (leitura do `.env`), `log.py` (log em JSON), `llm.py` (a chamada HTTP, a resposta estruturada, as novas tentativas e o modo sem rede), `vetores.py` (álgebra de similaridade, independente de domínio) e os testes desses módulos. A arquitetura inteira — validação de entrada, chamada validada, erro tratado, log estruturado — é justamente o que não muda; o que muda é o vocabulário das colunas.

E o `scripts/gravar-fixtures.py` ? Ele resolve o modo sem rede **depois** do trabalho, não no lugar dele: o script gera ordens a partir do dataset pelo código do projeto e grava as respostas reais do modelo em `data/fixtures/`. Enquanto `dados.py` e `cliente_llm.py` não lerem o dataset novo, ele não tem o que gravar. Adaptado o código, ele volta a valer:

```
python scripts\gravar-fixtures.py --n 25
```

Um cuidado de horário: rode-o num momento calmo do modelo. Gravado durante um pico de demanda, ele registra as respostas de erro (HTTP 503) como se fossem respostas normais — e o modo sem rede passa a repeti-las. Se isso acontecer, apague os arquivos de `data/fixtures/` e rode de novo.

O custo honesto: para quem já programa, um dia de trabalho — as adaptações da tabela acima são mecânicas, e a disciplina seguinte agradece o dataset próprio. Para quem está chegando agora a Python, o custo é maior do que o benefício: o caminho padrão existe para isso, e a compreensão demonstrada por escrito vale a mesma nota.

5. Como voltar atrás

Volta-se do mesmo jeito que se foi: rodando o `scripts/baixar-dados.py` original para retomar o dataset do padrão, restaurando as adaptações da seção 4 (o `git` mostra exatamente o que mudou — `git diff` lista arquivo a arquivo, e `git checkout -- <arquivo>` devolve a versão anterior de cada um) e confirmando com o `verifica-entrega.py`. Ninguém pergunta por quê, e ninguém perde nota — a regra dos dois caminhos com a mesma nota vale para a troca nos dois sentidos.

6. O aviso sobre o AI4I

Dois documentos do curso falam do AI4I 2020 em tom diferente, e nenhum dos dois está errado. O `MAPA-DE-DATASETS` desaconselha o AI4I a quem escolhe o caminho com dataset próprio **na turma padrão**: ali ele é o dado do exemplo, e trocar de domínio para continuar no dado do exemplo não é trocar de nada. Neste pacote, o AI4I é o **ponto de partida** — o estado pronto que garante o Contrato de Saída enquanto o aluno ainda não tem dataset próprio. A regra prática que fecha a diferença: usar o AI4I como fundação enquanto segue este guia é o caminho padrão deste pacote; adotá-lo **no lugar** do dataset do próprio domínio depois de ter trocado é voltar atrás — permitido, e a seção 5 diz como.